

move_iterator<T*> should be a random access iterator

Document #: P2520R0
Date: 2022-01-07
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[1.1 C++20 iterator improvements](#)

[1.2 Why does this matter](#)

[1.3 What does single-pass actually mean?](#)

[2 Proposal](#)

[3 References](#)

§ 1 Introduction

`std::move_iterator<Iter>` was added in C++11 as an iterator adaptor ([N1771]) that wraps `Iter` and changed only its `operator*()`, such that dereferencing a `std::move_iterator<Iter>` would give you an rvalue reference if dereferencing an `Iter` gave you an lvalue reference. Originally, the `iterator_category` of a `move_iterator<Iter>` was simply propagated from `Iter`'s (so `std::move_iterator<int*>::iterator_category` was `random_access_iterator_tag`), but then there was some discussion about whether it should instead *always* be `input_iterator_tag`. That discussion was resolved in [LWG1211] in favor of keeping the iterator category stronger, for performance reasons.

Howard Hinnant's example from that issue was:

```
vector<A> v;  
// ... build up a large vector of A ...  
vector<A> temp;  
// ... build up a large temporary vector of A to later be inserted ...  
typedef move_iterator<vector<A>::iterator> MI;  
// Now insert the temporary elements:  
v.insert(v.begin() + N, MI(temp.begin()), MI(temp.end()));
```

A major motivation for using `move_iterator` in the above example is the expectation that `A` is cheap to move but expensive to copy. I.e. the customer is looking for *high performance*. If we allow `vector::insert` to subtract two `MI`'s to get the distance between them, the customer enjoys

substantially better performance, compared to if we say that `vector::insert` can not subtract two MI's.

I can find no rationale for not giving this performance boost to our customers. Therefore I am strongly against restricting `move_iterator` to the `input_iterator_tag` category.

As a result, `std::move_iterator<T*>` is still, today, a C++17 random access iterator.

However, the adoption of the One Ranges paper [[P0896R4](#)] changed things a little.

`std::move_iterator<Iter>` was adjusted to use the new `iter_move` customization point and also introduced a new `iterator_concept` type alias, the new customization for C++20 iterators. As a result, we're in a little bit of an odd state because `std::move_iterator<T*>` is still a C++17 random access iterator (`std::move_iterator<T*>::iterator_category` is still `std::random_access_iterator_tag` and it has all the other operations), but it is only a C++20 input iterator (because `std::move_iterator<T*>::iterator_concept` exists and is only `std::input_iterator_tag`, `std::move_iterator<T*>` only satisfies `std::input_iterator` and not even `std::forward_iterator`).

This difference is a bit jarring, since usually whenever some iterator has different C++17 and C++20 categories, the C++20 category is the *stronger* of the two (e.g. `views::iota(0, 10)` is a C++20 random access range, but only a C++17 input range because its reference type is not a true reference). This is the one case in the standard library where the C++20 category is the *weaker* of the two.

§ 1.1 C++20 iterator improvements

There are several C++20 iterator improvements worth noting here before I get into the crux of the issue. One of the problems in the original C++17 model is that several operations were tied into the iterator category - that has now been split. Notably:

- you could only subtract two iterators, as in `e - b`, if they were random access iterators
- the only way to get the *size* of a range was to subtract iterators

Both of those problems are no longer exist in C++20. We now have `sized_sentinel_for`, and an iterator `I` can model `sized_sentinel_for<I>` even if it's an input iterator. This allows generic code to determine the distance between two iterators by simply subtracting them. Additionally, we have ranges now, and a range can be a `sized_range` (giving you its size cheaply) even if it's only an input range and even if its iterator do not model `sized_sentinel_for`.

§ 1.2 Why does this matter

The issue here is similar to Howard's original example in that library issue. Between the adoption of `views::move` under some name ([[P2446R1](#)]) and `ranges::to` ([[P1206R6](#)]), users will be able to conveniently produce new containers on the fly. If they write:

```
some_sized_range | views::move | ranges::to<vector>()
```

Then this is still okay - `views::move(some_sized_range)` is still sized, so the vector still only has to do a single allocation even if the resulting range is just an input range. This isn't really a problem.

But in the weaker case:

```
some_unsized_forward_range | views::move | ranges::to<vector>()
```

Now, suddenly, we can't do a single allocation - now we're trying to construct a vector from a non-sized input range, so the algorithm reduces to push_back in a loop. That's... not great, and it would be nice to avoid.

The standard library *could* do better. In fact, even outside of the standard library, user code could do better too. We could, for instance, recognize a range as being some kind of `ranges::move_view<R>` and simply treat it as an `R` (there's a `.base()` member function for this) for the purposes of determining whether we can easily figure out the size. That is, subvert the range model by simplifying special casing `move_view` in algorithms. This... is a thing that people could do, but doesn't really seem especially great? If the model needs to be subverted, perhaps it's the wrong model?

§ 1.3 What does single-pass actually mean?

The question really boils down to: is `move_iterator<T*>` a single-pass iterator or not? The text we have in the standard, in 23.3.4.11 [\[iterator.concept.forward\]](#) (and also in 23.3.5.5 [\[forward.iterators\]](#)) is:

4 Two dereferenceable iterators `a` and `b` of type `X` offer the *multi-pass guarantee* if:

(4.1) — `a == b` implies `++a == ++b` and

(4.2) — the expression `((void)[])(X x){++x;}(a), *a` is equivalent to the expression `*a`.

`move_iterator<T*>` very much satisfies both of those requirements. This issue here is not dereferencing. Given a `move_iterator<T*>`, this logic is fine:

```
void f(move_iterator<T*> a) {  
    auto&& x = *a;  
    auto&& y = *a;  
    auto&& z = *a;  
}
```

This is fine. Nothing actually happens - no move occurs. We're simply binding three different rvalue references to the same underlying object. On the other hand, this is *not* fine:

```
void f(move_iterator<T*> a) {  
    auto x = *a;  
    auto y = *a; // oops, double-move  
}
```

It would appear that the fact that we can't even dereference the same iterator twice (depending on what we do with the result) would argue against the fact that this can be considered to be a multi-pass iterator.

But it's also worth point out that the same holds true of *any* range whose reference type is an rvalue reference type. With C++20, we can construct those easily enough:

```
void f(vector<string> words) {  
    // currently (per P2446) this is a C++20 input range, whose reference  
    // type is string&&  
}
```

```

auto r1 = words | views::move;

// currently this is a C++20 random access range, whose reference type
// is string&&
auto r2 = words | views::transform([](string& s) -> string&& { return
    std::move(s); });
}

```

In the above, `r1` and `r2` are really the same range - they yield the same kinds of elements. `r1` is both a lot shorter to declare and conveys the intent more directly, but it's also just an input range. But if we have a problem with `move_iterator<T*>` being a random access iterator because it gives you a `T&&` which you can move out of, thus disallowing certain multi-pass algorithms... then surely we should have just as much a problem with *any* range whose reference type is an rvalue reference type? This is statically detectable after all. But we don't do that, `r2` is still random access. Which I think is correct.

There are algorithms that aren't usable with `move_iterators`. For example, `std::ranges::min(words | views::move)` is very ill-advised: you will get the smallest string in the collection, but you will also move out all of the rest, leaving you a range of empty strings. Probably not what the user would've wanted to happen. But there isn't anything about `words | views::move` that fails to meet the requirements of `std::ranges::min` - and `std::ranges::min` doesn't even require `forward_range`, anyway. Even algorithms that do require `forward` or better that might give questionable answers with `r1` would very much give those same questionable answers with `r2`. Any problem that would nominally be introduced by allowing `move_iterator` to be a random access iterator already exists under the more cumbersome syntax. It's just that the cumbersome syntax offers better performance in certain contexts, and I don't want to have to be put in a position to have to ever recommend it.

Ultimately, it's not clear to me why `move_iterator<T*>` needs to be a C++20 input iterator – which either pushes the burden to library authors everywhere to have to recognize `move_iterator` and work around its deficiencies or pushes the burden onto users to write `views::transform` instead, rather than simply making `move_iterator` properly advertise its capabilities.

§ 2 Proposal

Change `std::move_iterator<Iter>::iterator_concept` to be `Iter's iterator_concept`.

In 23.5.3.2 [\[move.iterator\]](#):

```

namespace std {
    template<class Iterator>
    class move_iterator {
    public:
        using iterator_type      = Iterator;
        - using iterator_concept = input_iterator_tag;
        + using iterator_concept = see below;
        using iterator_category = see below;           // not always
            present
        using value_type         = iter_value_t<Iterator>;
        using difference_type    = iter_difference_t<Iterator>;
        using pointer            = Iterator;
        using reference          = iter_rvalue_reference_t<Iterator>;
    };
}

```

- 0 The member *typedef-name* `iterator_concept` is defined as follows:
- (0.1) — If `Iterator` models `random_access_iterator`, then `iterator_concept` denotes `random_access_iterator_tag`.
 - (0.2) — Otherwise, if `Iterator` models `bidirectional_iterator`, then `iterator_concept` denotes `bidirectional_iterator_tag`.
 - (0.3) — Otherwise, if `Iterator` models `forward_iterator`, then `iterator_concept` denotes `forward_iterator_tag`.
 - (0.4) — Otherwise, `iterator_concept` denotes `input_iterator_tag`.

1 The member *typedef-name* `iterator_category` is defined if and only if [...]

Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define __cpp_lib_move_iterator_concept 20XXXXL // also in <iterators>
```

§ 3 References

[LWG1211] Alisdair Meredith. Move iterators should be restricted as input iterators.

<https://wg21.link/lwg1211>

[N1771] H. Hinnant, D. Abrahams, P. Dimov, D. Gregor, A. Hommel, A. Meredith. 2005-03-03. Impact of the rvalue reference on the Standard Library.

<https://wg21.link/n1771>

[P0896R4] Eric Niebler, Casey Carter, Christopher Di Bella. 2018-11-09. The One Ranges Proposal.

<https://wg21.link/p0896r4>

[P1206R6] Corentin Jabot, Eric Niebler, Casey Carter. 2021-08-03. Conversions from ranges to containers.

<https://wg21.link/p1206r6>

[P2446R1] Barry Revzin. 2021-11-17. `views::all_move`.

<https://wg21.link/p2446r1>